SentinelAI: An Adaptive Cyber Defence Platform with Explainable Intrusion Detection and Q-Learning Honeypot Intelligence

Dr. Bhagyashree Ambore, M. Sri Snigdha, M. H. Ullasa, Jayanth M, Kumari Pallavi
Department of Information Science and Engineering
RNS Institute of Technology
Bengaluru, India

***Abstract*—This paper presents SentinelAI, an adaptive cyber-defence platform designed toward autonomous and self-evolving operation that integrates dynamic threat detection, reinforcement-learning-driven honeypot deception, and a live, full-stack operator platform. Current intrusion detection and prevention systems remain largely reactive and rule-bound, leaving them poorly equipped against zero-day exploits and advanced persistent threats (APTs). SentinelAI's full proposed design comprises five engines: (1) a Multi-Model Threat Detection Engine combining Random Forest, XGBoost, 1D-CNN, and Isolation Forest for ensemble anomaly detection; (2) an Adaptive Honeypot Intelligence Engine, designed around Proximal Policy Optimization (PPO) for dynamic decoy reconfiguration; (3) a DRL Response Engine based on Deep Q-Networks (DQN) for autonomous defensive action selection; (4) a Cross-Domain Threat Intelligence Engine using BERT-based NLP to extract and correlate threat indicators; and (5) a Self-Evolution Module using meta-learning with Elastic Weight Consolidation (EWC) to enable continuous model adaptation. Beyond this architectural design and a formal problem formulation, this paper reports a functional, integrated four-service implementation (frontend, backend, AI inference engine, and adaptive honeypot), validated end-to-end: a Random Forest classifier trained on real CIC-IDS2017 traffic achieves 98.43% accuracy and 0.9862 weighted F1, and a tabular Q-learning agent recommends adaptive honeypot profiles based on live attacker engagement, in place of the originally proposed PPO policy for the current, discrete-action-space phase of the system. The DQN-based response engine, BERT-based threat intelligence pipeline, deep-sequence detection tier, and GAN/EWC self-evolution mechanism remain future work and are reported honestly as such rather than as projected results. Closing the autonomous-response loop and completing real-data coverage of the platform's eleven-class attack taxonomy are identified as the highest-priority next steps.**

***Index Terms*—SentinelAI, Autonomous Cyber Defence, Adaptive Honeypot, Reinforcement Learning, Intrusion Detection, Zero-Day Attack, Threat Intelligence, Deep Learning, Self-Evolving Security, Security Orchestration**

I. Introduction

Today's threat landscape has outpaced what traditional defence architectures can handle. Advanced persistent threats (APTs), polymorphic malware, and automated exploit frameworks adapt to defensive countermeasures in near-real time, exposing the structural limits of signature-based Intrusion Detection Systems (IDS), static honeypots, and human-operated Security Operations Centers (SOCs) [3]. These are not incidental shortcomings; they are fundamental constraints baked into how those systems operate.

The root problem is dependence on human analytical bandwidth. Even the most sophisticated SIEM platform remains a tool that assists analysts rather than replacing them. That constraint produces a detect-analyze-respond cycle measured in minutes to hours, while automated attacks execute their objectives in seconds [1]. The average time to identify and contain a breach exceeds 200 days [21], and attacker dwell times in enterprise networks continue to run in the multi-week range [22]. Closing that gap requires systems capable of autonomous detection, decision-making, and response — with no human in the critical path.

Honeypot technology presents its own version of this problem. Its theoretical value as a deception and intelligence-collection mechanism is well established, but static configurations give sophisticated adversaries sufficient signal to identify and avoid them within seconds of contact [24]. The adversaries most worth deceiving are precisely those most capable of seeing through static deployments [5].

This paper presents SentinelAI, an integrated platform design that addresses these limitations through five tightly coupled components: (i) adaptive honeypot intelligence driven by reinforcement learning; (ii) ensemble multi-model threat detection spanning supervised, deep, and unsupervised learning paradigms; (iii) autonomous RL-driven response orchestration; (iv) cross-domain NLP-assisted threat intelligence; and (v) meta-learning-based self-evolution. The system targets enterprise network environments and is designed for evaluation against established benchmark datasets.

The primary contributions of this paper are:

- Design of a unified cyber defence architecture that integrates adaptive honeypots, multi-model threat detection, autonomous response, and continuous learning into a single self-evolving platform.
- A reinforcement learning formulation for dynamic honeypot reconfiguration, modelled as a finite Markov Decision Process, with a reward function specifically designed to maximise attacker engagement and intelligence collection.
- An ensemble detection pipeline specification combining supervised, deep sequence, and unsupervised anomaly

detection components with a meta-learner stacking layer.
- A meta-learning-based self-evolution mechanism using Elastic Weight Consolidation and GAN-based data augmentation to enable continuous adaptation without catastrophic forgetting.
- A formal problem formulation and a validated preprocessing pipeline, currently applied to CICIDS2017 and designed to generalise to six further benchmark intrusion detection datasets.
- A functional, integrated four-service implementation (frontend, backend, AI inference engine, and adaptive honeypot) going beyond the frontend-only proof-of-concept reported in earlier work, including a Random Forest detection model validated on real CICIDS2017 traffic and a Q-learning-based honeypot profile recommendation mechanism with a persistent, auditable policy.

II. Literature Review

A. AI-Driven Threat Detection and Intelligence

Machine learning for network intrusion detection has advanced well beyond early statistical methods, with deep and ensemble approaches now dominant. Garg et al. [1] propose an adaptive AI-driven threat intelligence framework for proactive cyber defence in dynamic network environments, demonstrating that predictive correlation of threat indicators can substantially narrow the window of exposure to emerging campaigns. Rao and Kumar [2] complement this with a system for monitoring and countering underground economy operations, arguing that effective threat intelligence must extend beyond perimeter telemetry to cover adversarial infrastructure.

Zero-day attack mitigation remains an open problem. Ponaganti et al. [3] address it through predictive behavioral analysis, demonstrating that behavioral profiling of network entities can surface anomalies indicative of novel exploits before signature databases are updated. Ring et al. [26] provide a systematic survey of network-based intrusion detection datasets, establishing the evaluation benchmarks that inform the experimental methodology in this paper.

Apruzzese et al. [25] critically assess the effectiveness of machine and deep learning for cybersecurity, noting that while classification accuracy on benchmark datasets is high, real-world deployment introduces distributional shift challenges requiring continuous model adaptation — a limitation directly addressed by SentinelAI's self-evolution module.

B. Reinforcement Learning for Cyber Defence

Reinforcement learning is well-suited to autonomous cyber defence because it frames defence as a sequential decision problem under uncertainty. Abbas and Asghar [4] demonstrate an adaptive RL-based response setup in which policy gradient methods learn effective response strategies against evolving attack patterns in simulated network environments. Babar et al. [19] contribute an RL environment specifically designed for training network defence agents, providing a reproducible setup for agent evaluation.

Loevenich et al. [17] identified reward engineering, adversarial manipulation, and operational robustness as critical issues affecting the deployment of autonomous defence agents in real-world coalition networks. These concerns directly influenced the design of SentinelAI's Self-Evolution Module, which incorporates adversarially robust learning mechanisms. Théron and Kott [20] earlier envisioned future cyber conflicts involving autonomous defensive agents operating against intelligent autonomous malware, a scenario increasingly relevant as AI capabilities advance.

C. Honeypot Technology and Adaptive Deception

Traditional honeypots are valuable for passive intelligence collection, but their static configuration presents a fundamental limitation. Holz and Raynal [24] document honeypot fingerprinting techniques, demonstrating that fixed deployments are exposed through timing analysis, anomalous service responses, and artifact inspection. This vulnerability motivates SentinelAI's use of reinforcement learning for adaptive reconfiguration in place of static deployment.

Tiwari and Kumar [5] proposed a blockchain-supported distributed honeypot framework combining fuzzy logic with dynamic management techniques to improve resilience and attacker engagement. Malarvizhi et al. [6] demonstrated that AI-powered honeypots capable of realistic service emulation can capture higher-quality behavioral intelligence from attackers. Alosaimi and Alatawi [7] reported that adaptive reconfiguration mechanisms significantly improve resistance to fingerprinting attempts compared with conventional static deployments.

For real-time malware classification based on honeypot data, Faísco et al. [10] demonstrate that artifacts collected by honeypots can feed automated malware classification pipelines, creating a feedback loop in which deception and detection reinforce each other.

D. Ensemble and Deep Learning for Intrusion Detection

Ensemble methods consistently outperform single-model baselines in IDS benchmarks. Altrad et al. [11] deploy a Random Forest-based intrusion detection system for zero-day attack mitigation in industrial control systems, demonstrating that ensemble diversity is particularly effective for generalising to new attack types. Wang et al. [12] develop a real-time DDoS detection system using one-dimensional deep convolutional neural networks, showing that 1D-CNN architectures are effective for sequential packet-level feature extraction.

For brute-force attack detection, Bouriche et al. [13] employ neural hidden Markov models in a semi-supervised setup, reducing the quantity of labeled data required for deployment. Htet et al. [14] and Zahreldine et al. [15] introduce active-response systems for SSH brute-force attacks demonstrating real-time mitigation at the application layer.

E. Adversarial Robustness and Model Probing

As machine learning becomes increasingly integrated into cybersecurity systems, attacks targeting the defensive models

themselves have emerged as a significant concern. Wang et al. [16] investigated black-box model probing attacks, demonstrating how adversaries can exploit prediction probability scores to infer information about deployed machine learning models.

These findings highlight the importance of securing not only the network infrastructure but also the underlying AI components. SentinelAI accordingly incorporates an adversarial data validation mechanism designed to detect potentially malicious inputs before they can influence continual learning and retraining processes.

F. Self-Evolving and Generative Defence Architectures

Polinati [8] introduced Generative Adversarial Deception Networks (GADN), employing generative adversarial techniques to create synthetic threat data and enhance defensive capabilities in cloud-native environments. Tanvir et al. [9] demonstrated the effectiveness of self-supervised and domain-adaptive learning for detecting cyberattacks in healthcare IoT environments, showing that adaptation to new operational contexts significantly improves detection performance when deployment conditions differ from training environments.

SentinelAI's Self-Evolution Module builds on both of these directions, combining synthetic data generation, continual learning, and domain-adaptive fine-tuning to maintain effectiveness against emerging threats.

G. Threat Intelligence Extraction

Al-Yasiri et al. [18] proposed a conceptual framework for extracting structured security events from Cyber Threat Intelligence (CTI) feeds, enabling automated analysis of threat reports and security advisories. SentinelAI's NLP-driven threat intelligence pipeline builds upon this concept, extracting entities and relationships from unstructured security content and mapping them to the MITRE ATT&CK framework [23] to support threat attribution and attack correlation.

Table I presents a comparative analysis of key research works surveyed in this paper.

H. Research Gap and Positioning

Existing work demonstrates substantial progress on individual cyber defence problems: improved detection accuracy, RL-based response, adaptive honeypots, and threat intelligence extraction. However, no prior system integrates all five capabilities into a single self-evolving platform.

Current RL-based defence systems [4], [19] do not incorporate adaptive deception. AI-driven honeypot systems [6], [7] do not provide autonomous response orchestration. Self-evolving architectures [8] are not coupled to real-time detection pipelines. SentinelAI addresses this gap through a proposed five-engine integrated architecture, with a functional four-service implementation currently realising selected components of the architecture, each engine contributing a measurable and distinct function to the overall defence objective.

III. Threat Model

SentinelAI is designed to operate against a well-defined attacker model. The assumed adversary possesses the following capabilities:

- **Knowledge:** The attacker has partial knowledge of the target network topology but no prior knowledge of the SentinelAI deployment, its honeypot configurations, or its detection models.
- **Access:** The attacker operates from outside the enterprise perimeter and has established at least one foothold via phishing, credential compromise, or exploitation of a public-facing service.
- **Capability:** The attacker can conduct reconnaissance, lateral movement, data exfiltration, and denial-of-service campaigns. The attacker may attempt to identify and evade honeypots through active fingerprinting.
- **Adaptability:** The attacker is capable of modifying tactics, techniques, and procedures (TTPs) in response to observed defensive countermeasures, consistent with the APT threat model.

The system is *not* designed to defend against adversaries with complete knowledge of the deployed machine learning models or the ability to inject adversarial examples directly into the training pipeline. The adversarial data validation component in the Self-Evolution Module partially mitigates this risk, but complete adversarial ML robustness is identified as future work.

IV. Problem Definition and Objectives

A. Problem Formulation

Let a network environment $\mathcal{N}$ consist of a set of legitimate assets $\mathcal{L} = \{l_1, l_2, \ldots, l_n\}$ and a set of deployed honeypot nodes $\mathcal{H} = \{h_1, h_2, \ldots, h_m\}$. An attacker $\mathcal{A}$ executes an attack campaign as a sequence of actions $\mathcal{A} = \{a_1, a_2, \ldots, a_T\}$ with the objective of compromising one or more assets in $\mathcal{L}$.

The cyber defence problem is to design a system $\mathcal{D}$ that: (i) detects attacker actions with maximum sensitivity and minimum false positive rate; (ii) deploys deceptive environments to engage and study the attacker; and (iii) executes defensive responses to minimise attacker impact on $\mathcal{L}$.

Formally, $\mathcal{D}$ must maximise:

$$\mathcal{J}(\mathcal{D}) = \alpha \cdot \text{DR} + \beta \cdot \text{EI} - \gamma \cdot \text{FPR} - \delta \cdot \text{RT} \qquad (1)$$

where DR is the detection rate, EI is the average attacker engagement index (time spent in deceptive environments), FPR is the false positive rate, RT is the mean response time, and $\alpha, \beta, \gamma, \delta$ are weighting coefficients reflecting the relative priority of each objective.

B. Design Objectives

The target design objectives for the complete SentinelAI architecture are:

1) Achieve detection accuracy $\geq$98% with false positive rate $\leq$2% on multi-class network intrusion datasets.

TABLE I
COMPARATIVE ANALYSIS OF EXISTING RESEARCH WORKS

Sl.	Paper Title	Author	Year	Dataset	Technique	Key Metric	Limitations
1	Adaptive AI-Driven Threat Intelligence Framework for Proactive Cyber Defense	M. Garg et al.	2026	MITRE ATT&CK	NLP, Threat Correlation	Not reported	No real-time automated response
2	An Adaptive CTI System for Monitoring Underground Economy Operations	K. M. Rao and B. Kumar	2026	CTI Feeds	Threat Monitoring, CTI Analysis	Not reported	No autonomous mitigation
3	Adaptive AI Algorithms for Cyber Defense: Zero-Day Attack Mitigation	M. Ponaganti et al.	2025	NSL-KDD, CICIDS2017	ML, Behavioral Analysis	Acc. 97.3%	Limited scalability
4	Adaptive Cyber-defence: RL-based Response to Evolving Cyber-attacks	M. N. Abbas & M. N. Asghar	2025	Network Security Data	Reinforcement Learning	Reward convergence reported	Requires extensive training data
5	Adaptive Fuzzy Logic Framework for Distributed Honeypot Systems with Blockchain	A. Tiwari and D. Kumar	2025	Distributed Honeypot Logs	Fuzzy Logic, Blockchain	DR 94.1%	High implementation complexity
6	AI Powered Honeypots for Advanced Threat Detection and Analysis	K. Malarvizhi et al.	2025	Honeypot Interaction Data	AI-based Honeypots	Not reported	Resource intensive
7	Leveraging AI for Enhancing Real-Time Honeypot Security	R. Alosaimi & A. Alatawi	2025	Honeypot Traffic Data	Artificial Intelligence	Not reported	Limited evaluation scenarios
8	Generative Adversarial Deception Networks (GADN) for Cloud-Native Security	A. K. Polinati	2025	Cloud Security Data	GAN-based Deception	Not reported	Computational overhead
9	Self-Supervised Deep Learning for Cyber-Attack Detection in Healthcare IoT	M. I. M. Tanvir et al.	2025	Healthcare IoT Data	Self-Supervised DL	F1 96.8%	Domain-specific applicability
10	Towards Real-Time Malware Classification Through Honeypot Analysis	M. Faísco et al.	2025	Honeypot Malware Data	Malware Classification	Acc. 91.2%	Limited malware diversity
11	Random Forest-Based IDS for Zero-Day Attack Mitigation in ICS	A. Altrad et al.	2025	ICS Network Data	Random Forest	Acc. 98.1%, FPR 2.3%	Potential false positives
12	Real-time DDoS Detection Based on 1D Deep Convolutional Neural Network	Y. Wang et al.	2025	DDoS Traffic Dataset	1D-CNN	Acc. 99.1%	Requires large training datasets
13	Neural Hidden Markov Models for Semi-Supervised Brute Force Detection	M. Bouriche et al.	2025	Authentication Logs	Neural HMM	DR 95.4%	Detection latency
14	Brute Force Detection and Active Response for Web Login Using Wazuh	K. Z. Htet et al.	2025	Web Login Logs	Rule-based (Wazuh)	Not reported	Web login attacks only
15	Live Detection and Mitigation for SSH Brute Force Attacks	N. Zahreldine et al.	2025	SSH Traffic Logs	Live Detection and Mitigation	Not reported	Limited attack coverage
16	Detecting Black-Box Model Probing Attacks Through Probability Scores	Y. Wang et al.	2025	ML Model Query Data	Probability Score Analysis	AUC 0.94	Specific to AI model attacks
17	Towards Robust Autonomous Cyber Defense Agents in Coalition Networks	J. F. Loevenich et al.	2024	Coalition Network Simulation	Autonomous Defense Agents	Not reported	Early-stage implementation
18	A CTI Event Extraction Conceptual Model for Threat Intelligence Feeds	J. H. Al-Yasiri et al.	2024	CTI Feeds	Event Extraction Model	Not reported	Conceptual framework only
19	Towards Autonomous Network Defense: RL Environment for a Defense Agent	A. Babar et al.	2024	Simulated Network Env.	Reinforcement Learning	Not reported	Limited real-world validation
20	When Autonomous Goodware Will Fight Intelligent Malware	P. Théron and A. Kott	2019	N/A	Conceptual Study	N/A	Limited practical validation

2) Reduce mean detection-to-response time to below 5 seconds for high-severity threats.
3) Increase average attacker engagement duration in honeypot environments by $\geq$40% compared to static honeypot baselines.
4) Enable continuous autonomous improvement in detection accuracy through meta-learning self-evolution.
5) Provide interpretable, auditable decision trails for all automated defensive actions via SHAP-based explainability integration.

V. SYSTEM ARCHITECTURE

A. Architectural Overview

The SentinelAI architecture proposes five tightly coupled engines, as shown in Fig. 1. This is the complete proposed architecture rather than a claim that all five engines are currently deployed as independent services. The current implementation is a four-service system comprising the React/Vite frontend, Node.js/Express backend, Python/FastAPI AI engine, and Cowrie-based honeypot. The proposed orchestration and inter-engine communication mechanisms therefore represent the target architecture for future expansion.

The data flow through the system is as follows. Raw network traffic is ingested by the Data Ingestion Layer, which normalises and routes events to the Threat Detection Engine. Detection outputs are forwarded to both the Response Engine (for autonomous action) and the Threat Intelligence Engine (for enrichment and attribution). The Adaptive Honeypot Engine operates in parallel, redirecting suspected attacker traffic to decoy nodes and feeding interaction logs back to the Detection Engine for retraining. The Self-Evolution Module monitors model performance continuously and triggers incre-

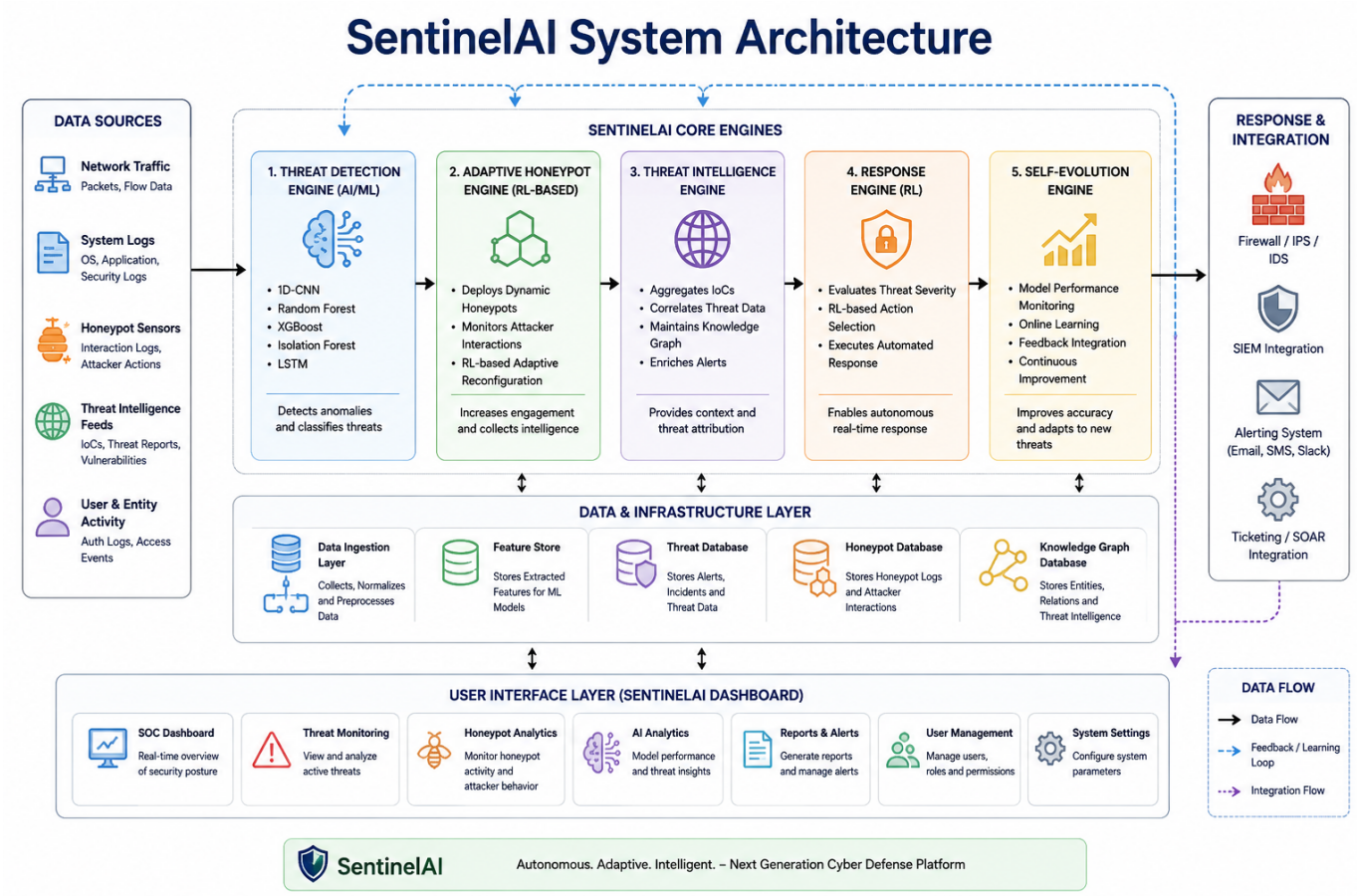


Fig. 1. SentinelAI system architecture illustrating the integration of traffic collection, AI-based threat detection, adaptive honeypot intelligence, threat intelligence analysis, reinforcement learning, autonomous response, and visualization components.

mental updates after each confirmed incident.

B. Security Architecture

Authentication for operator-facing services is implemented using JSON Web Tokens (JWT), with short-lived access tokens and Role-Based Access Control (RBAC) restricting access according to assigned roles (administrator, analyst, and viewer). These controls provide the security baseline for the current implementation. However, the current system should not be described as a complete Zero Trust architecture: continuous service authentication, comprehensive microsegmentation, and complete least-privilege enforcement across all service-to-service communication paths remain security-hardening objectives. Similarly, inter-service transport encryption is dependent on the deployment configuration and is not claimed as a fully enforced property of the current prototype. These limitations are consistent with the broader principle that a security platform must itself be protected against becoming an attack surface [8].

C. Current Implementation Status

The architecture described above represents the complete intended design of SentinelAI. In the current revision, the platform has progressed substantially beyond the frontend-only prototype reported in earlier work. A complete four-service system comprising a React/Vite frontend, Node.js/Express backend, Python/FastAPI AI engine, and Cowrie-based honeypot has been implemented, containerised, and integrated into an operational end-to-end pipeline.

Table II summarises the implementation status of the five proposed SentinelAI engines with respect to the architecture presented in Section V. The table explicitly distinguishes implemented functionality from components that remain partial or planned, thereby avoiding overstatement of the current system capabilities. Sections XI and X provide further evidence of the implemented prototype and describe the revised phase-by-phase development roadmap.

VI. Proposed Methodology

A. Multi-Model Threat Detection Engine

The detection engine employs a three-tier layered ensemble design [25]:

Tier 1 — Supervised Classification: Random Forest and XGBoost models perform multi-class attack classification. Random Forest provides interpretability and robustness to noisy features [11]. XGBoost addresses class imbalance

TABLE II
IMPLEMENTATION STATUS OF THE FIVE SENTINELAI ENGINES

Engine	Status	Current Realisation
Multi-Model Threat Detection	Partial	Random Forest (RF), XGBoost, Isolation Forest, and an SGD-based online learner are implemented and trained. RF is validated on CICIDS2017 data, achieving 98.43% accuracy and 0.9862 weighted F1. The proposed 1D-CNN/LSTM deep-sequence tier and learned meta-learner stacking are not yet implemented; fusion currently uses a fixed weighted heuristic.
Adaptive Honeypot Intelligence	Implemented (Q-learning)	A tabular Q-learning agent using an epsilon-greedy policy with decaying ϵ recommends honeypot OS/service profiles according to a reward function based on attacker engagement. The recommendation is persisted through the Q-learning policy, but the active Cowrie configuration is not automatically modified at runtime. PPO was not adopted in this phase because the currently realised honeypot action space is relatively small and discrete. Q-learning therefore provides an interpretable and sample-efficient policy.
DRL Response Engine	Not implemented	Response actions such as IP blocking are currently logged and reflected in the operator dashboard; however, network-layer enforcement through firewall or security-group actions is not yet executed. DQN-based autonomous response selection remains future work.
Cross-Domain Threat Intelligence	Partial	A structured MITRE ATT&CK mapping module automatically associates each detection with a corresponding technique identifier. An IP-reputation enrichment service based on AbuseIPDB has also been integrated. BERT-based extraction from unstructured cyber-threat-intelligence feeds is not yet implemented.
Self-Evolution Module	Partial	An SGD-based classifier and operator-feedback path support incremental learning through `partial_fit`, with a fixed holdout set available for monitoring. However, the supplied SGD artifact currently fails to deserialize in the containerised deployment because of a feature-function compatibility issue. GAN-based data augmentation and Elastic Weight Consolidation are not yet implemented.

through sample weight adjustment and contributes gradient-boosted ensemble diversity. Both models operate on engineered network-flow features including packet counts, byte counts, inter-arrival times, and protocol-level statistics. *This tier is implemented using scikit-learn [28] and XGBoost [29]; see Section XI for validated results.*

Tier 2 — Deep Sequence Learning: A 1D-CNN with LSTM layers processes raw packet byte streams and temporal traffic behaviour. The convolutional layers extract local byte-level features from payloads, while the LSTM captures sequential dependencies in the traffic time series [12]. *This tier is part of the proposed design and has not yet been implemented; it remains future work (Section XII).*

Tier 3 — Anomaly Detection: Isolation Forest [27] and an autoencoder independently produce anomaly scores for traffic that does not conform to known attack or benign signatures, enabling detection of zero-day and previously unseen attacks [3]. *The Isolation Forest component is implemented and integrated into live inference; the autoencoder component is not yet implemented.*

A meta-learner (stacking ensemble) aggregates all tier outputs into a single threat label and confidence score. If any tier flags traffic as malicious with confidence exceeding threshold τ_{high}, the system immediately escalates to the Response Engine. Confidence scores between τ_{low} and τ_{high} are escalated to a human analyst. *In the current implementation this fusion step is realised as a fixed weighted combination of the classifier confidence and the anomaly score rather than a trained meta-learner; learning the fusion weights is identified as near-term future work.*

B. Adaptive Honeypot Intelligence Engine

In the proposed architecture, the Adaptive Honeypot Engine manages a pool of honeypot nodes whose service profiles can be reconfigured in real time. Each node exposes tunable parameters including OS fingerprint, open port selection, service banner content, response timing, and file system artifacts. The full proposed design uses a PPO-based policy network to continuously adjust these parameters in response to observed attacker behaviour. The motivation for adaptive deception is grounded in documented honeypot fingerprinting weaknesses [24], which demonstrate that static deployments can be identified by skilled attackers.

The honeypot reconfiguration problem is formulated as a finite Markov Decision Process:

$$\mathcal{M}_{HP} = (\mathcal{S}_{HP}, \mathcal{A}_{HP}, \mathcal{T}_{HP}, \mathcal{R}_{HP}, \gamma) \tag{2}$$

The state space $\mathcal{S}_{HP}$ encodes the attacker behaviour fingerprint (commands issued, tools used, and probing patterns), interaction duration, fingerprinting attempt count, and current honeypot configuration. The action space $\mathcal{A}_{HP}$ encompasses OS profile switching, service banner rotation, timing jitter injection, and artifact planting. The reward function $\mathcal{R}_{HP}$ assigns positive rewards for extended attacker engagement and negative rewards for detected honeypot identification events.

PPO is selected in the full proposed design for policy optimisation because of its suitability for more complex and higher-dimensional action spaces relative to tabular approaches [4]:

$$\mathcal{L}^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t \right) \right] \tag{3}$$

where

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \tag{4}$$

is the probability ratio and $\hat{A}_t$ is the estimated advantage.

Current implementation. Because the honeypot action space realised so far is small and discrete, corresponding to a fixed set of OS/service profile recommendations, the present implementation uses tabular Q-learning with an ϵ-greedy exploration policy and ϵ-decay in place of PPO:

$$Q(s,a) \leftarrow Q(s,a) + \alpha_{lr}\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{5}$$

The Q-learning agent selects and records a recommended profile based on the observed attacker state and reward signal. The recommendation is not automatically applied to the active Cowrie instance in the current implementation. This represents a deliberate scope reduction from the full proposed architecture while preserving the underlying MDP formulation. Migration to PPO and automatic runtime profile reconfiguration are identified as future work for a larger or more complex action space (Section XII).

C. Deep Reinforcement Learning Response Engine

The Response Engine employs a DQN agent that maps threat classifications directly to defensive actions. The action space includes: blocking source IPs at the firewall, rate-limiting traffic flows, resetting suspicious connections, redirecting traffic to honeypots, isolating hosts to a VLAN, escalating alerts to a SOAR platform, and initiating automated forensic capture. The RL environment design follows the autonomous network defence agent framework of Babar et al. [19].

The Q-network approximates the optimal action-value function:

$$Q(s, a; \theta) \approx Q^*(s, a) \tag{6}$$

Training employs an experience replay buffer of (s, a, r, s') tuples with a target network for stable gradient estimation:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\left(y_j - Q(s_j, a_j; \theta)\right)^2\right] \tag{7}$$

where $y_j = r_j + \gamma \max_{a'} Q(s_{j+1}, a'; \theta^-)$ and θ^- denotes the target network parameters. The reward function is designed to minimise attacker dwell time and asset impact:

$$r_t = w_1 \cdot \Delta\text{ImpactScore} - w_2 \cdot \text{FalsePositive} - w_3 \cdot \text{ResponseLatency} \tag{8}$$

Current implementation. The DQN response engine described above remains a proposed design. In the current system, a high-confidence detection produces a recommended action (`BLOCK`/`MONITOR`/`ALLOW`) that is logged, mapped to MITRE ATT&CK, and surfaced to the operator dashboard in real time via WebSocket; an operator-initiated block updates the asset's status and is broadcast to connected clients. Actual network-layer enforcement (e.g., a firewall or cloud security-group rule change) is not yet wired to this decision, so the response loop is presently decision-support rather than fully autonomous. Closing this loop is the highest-priority item in the roadmap (Section X, Phase 6).

D. Threat Intelligence Engine

The Threat Intelligence Engine employs a BERT-based NLP model fine-tuned on cybersecurity corpora to extract structured intelligence from unstructured sources including security advisories, vulnerability disclosures, and threat actor reports. The extraction design follows the conceptual model of Al-Yasiri et al. [18]. Extracted entities — CVE identifiers, threat actor attributions, malware family names, and indicators of compromise (IoCs) — are stored in a graph database capturing inter-entity relationships.

An LLM-assisted correlation step maps observed network events to known threat actor TTPs within the MITRE ATT&CK framework [23], supporting both threat attribution and predictive modelling [2].

Current implementation. The BERT-based unstructured CTI extraction pipeline described above remains future work. A structured, rule-based mapping from detected attack classes to MITRE ATT&CK technique identifiers is implemented and applied automatically to every stored detection, and an IP-reputation enrichment step (via the AbuseIPDB API) was added to the pipeline as a lightweight, complementary threat-intelligence signal not present in the original design.

E. Self-Evolution Module

The Self-Evolution Module implements a meta-learning framework enabling SentinelAI to improve defensive performance through operational experience, building on the GADN architecture of Polinati [8] and domain-adaptive learning demonstrated by Tanvir et al. [9]. Following each confirmed security incident, the module: (i) analyses detection outcomes to identify model failure modes; (ii) generates synthetic adversarial training samples using GAN-based data augmentation; (iii) performs incremental model fine-tuning on the augmented dataset; and (iv) updates honeypot configuration policies based on observed attacker behaviour patterns.

Catastrophic forgetting during incremental training is mitigated through Elastic Weight Consolidation (EWC) [17]:

$$\mathcal{L}_{EWC}(\theta) = \mathcal{L}_{new}(\theta) + \frac{\lambda}{2} \sum_i F_i(\theta_i - \theta_i^*)^2 \tag{9}$$

where F_i is the Fisher information matrix diagonal and θ_i^* are the parameters from prior training. A separate input validation classifier screens training samples for adversarial manipulation before admission to the fine-tuning pipeline [16], [17].

Current implementation. The GAN-augmentation and EWC-regularised fine-tuning pipeline above is future work. A simpler continual-learning mechanism has been implemented through an SGD-based linear classifier with a `partial_fit` feedback interface for operator-verified labels. The AI engine also maintains a fixed holdout set for monitoring online-model performance. However, the supplied containerised deployment currently fails to deserialize the saved SGD artifact because it references the `log_transform_features` function incompatibly. Consequently, the continual-learning mechanism should be considered partially implemented and not fully operational in the current containerised deployment until the model-artifact compatibility issue is resolved. The adversarial input-validation classifier described above has not yet been implemented.

Algorithm 1 Proposed Multi-Model Threat Detection Pipeline

Require: Network flow f, available model components $\{RF, XGB, CNN, IF, AE\}$
Ensure: Threat classification c, confidence score σ

Extract feature vector $\mathbf{x} \leftarrow$ FeatureExtract(f)
$p_1 \leftarrow$ RF.predict_proba$(\mathbf{x})$
$p_2 \leftarrow$ XGB.predict_proba$(\mathbf{x})$
$p_3 \leftarrow$ CNN.forward$(f$.bytes$)$
$a_1 \leftarrow$ IF.anomaly_score$(\mathbf{x})$
$a_2 \leftarrow$ AE.reconstruction_error$(\mathbf{x})$
$\mathbf{p}_{ensemble} \leftarrow$ MetaLearner$(p_1, p_2, p_3, a_1, a_2)$
$c \leftarrow \arg\max(\mathbf{p}_{ensemble})$
$\sigma \leftarrow \max(\mathbf{p}_{ensemble})$
if $\sigma > \tau_{high}$ **and** $c \neq$ BENIGN **then**
  TriggerResponse(f, c, σ)
else if $\sigma > \tau_{low}$ **and** $c \neq$ BENIGN **then**
  EscalateToAnalyst(f, c, σ)
end if
return c, σ

VII. Algorithms

A. Threat Detection Algorithm

Note on current implementation: Algorithm 1 specifies the complete proposed multi-model detection pipeline. In the current implementation, the CNN component p_3 and autoencoder component a_2 are not yet computed, and the proposed trained stacking meta-learner is not yet implemented. The current fusion step uses a fixed weighted combination of the implemented classifier confidence and Isolation Forest anomaly score. Random Forest and Isolation Forest are executed in the live inference path, while the available XGBoost model is evaluated separately and its reported result is based on synthetic training data. The remaining deep-learning and learned-stacking components are future work.

B. Adaptive Honeypot Profile Recommendation Algorithm

Algorithm 2 Honeypot Profile Recommendation via Q-Learning

Require: Attacker interaction log $\mathcal{I}$, Q-table Q
Ensure: Recommended honeypot profile $\mathcal{C}_{recommended}$

$s \leftarrow$ BuildState$(\mathcal{I})$
Detect fingerprinting attempts $\mathcal{F} \subset \mathcal{I}$
if $|\mathcal{F}| > \tau_{fp}$ **then**
  $a \leftarrow \epsilon$-greedy action selection over $Q(s, \cdot)$
  $\mathcal{C}_{recommended} \leftarrow$ SelectProfile(a)
  $r \leftarrow$ ComputeReward$(\mathcal{I}, a)$
  Update $Q(s, a)$ using the temporal-difference rule
  Persist updated Q-table
else
  $\mathcal{C}_{recommended} \leftarrow \mathcal{C}_{current}$
end if
return $\mathcal{C}_{recommended}$

Implementation note: The current Q-learning agent recommends an alternative honeypot profile based on attacker engagement and fingerprinting behaviour. The recommendation is recorded for operator use, but the active Cowrie configuration is not automatically modified at runtime. Automatic application of timing jitter, banner rotation, OS-profile switching, and other deception actions remains future work.

Algorithm 2 is implemented and operational in the current system using the tabular Q-learning update of Eq. (4) in place of the proposed PPO objective of Eq. (3), for the reasons discussed in Section VI-B. A persistent Q-table is maintained across honeypot restarts.

VIII. Dataset and Preprocessing Pipeline

A. Datasets

The SentinelAI framework is designed for evaluation against seven widely recognised cybersecurity benchmark datasets, summarised in Table III. Dataset selection follows the recommendations of Ring et al. [26], ensuring coverage of both traditional and contemporary cyberattack scenarios across enterprise and IoT environments. Of these seven, CICIDS2017 is the dataset currently integrated into the training pipeline; the remaining six are selected candidates for the dataset-expansion phase of future work (Section XII).

TABLE III
Benchmark Dataset Summary and Integration Status

Dataset	Year	Records	Status
NSL-KDD	2009	148,517	Candidate (future work)
UNSW-NB15	2015	2,540,044	Candidate (future work)
CICIDS2017	2017	2,830,743	**In use (partial)**
CICIDS2018	2018	7,322,185	Candidate (future work)
CIC-DDoS2019	2019	56,863,581	Candidate (future work)
TON-IoT	2020	461,043	Candidate (future work)
Bot-IoT	2018	73,370,762	Candidate (future work)

CICIDS2017 is designated as the primary dataset for multi-class intrusion detection experiments due to its realistic traffic patterns and broad attack category coverage. In the current implementation, four of the eight official CICIDS2017 capture files are integrated (Tuesday brute-force, and Friday botnet, port-scan, and DDoS captures), covering six of the eleven attack classes in the platform's target taxonomy (BENIGN, SSH Brute Force, Credential Stuffing, Port Scan, DDoS, and Malware Beacon) with real labelled traffic. The remaining five classes (SQL Injection, XSS Attempt, RCE Attempt, Lateral Movement, and Data Exfiltration) are defined in the taxonomy and supported end-to-end by the pipeline, but do not yet have real CICIDS2017 training examples; integrating the remaining official CICIDS2017 files (Monday, Wednesday, and both Thursday captures) to populate these classes is the immediate next step identified in the roadmap. CIC-DDoS2019 remains planned for dedicated DDoS-detection evaluation and UNSW-NB15 for anomaly-detection generalisation analysis, consistent with established evaluation practice in the intrusion detection community [11], [12].

B. Preprocessing Pipeline

A standardised preprocessing pipeline, implemented in the AI engine's feature-extraction module, is applied to the integrated dataset and is designed to generalise to the remaining candidate datasets. Raw CICIDS2017 flow records, produced by CICFlowMeter from packet captures, are ingested directly in CSV form.

The implemented preprocessing procedure comprises the following stages:

1) Column-name normalisation and fuzzy matching against the platform's canonical feature schema, so that minor header variations between dataset releases do not silently break ingestion.
2) Removal of invalid records containing infinite or NaN values arising from division-by-zero operations during flow statistic computation (e.g. `Flow Bytes/s` when flow duration is zero).
3) Standardisation (zero-mean, unit-variance scaling) of numerical attributes within a scikit-learn `Pipeline`, ensuring the same transform is applied consistently at training and inference time.
4) Label harmonisation, mapping each dataset's raw attack labels onto the platform's eleven-class target taxonomy.

Two elements of the originally proposed pipeline are not yet implemented: Min-Max normalisation has been realised as standard scaling instead (a design choice better suited to the tree-based models currently used), and SMOTE-based minority-class oversampling has not yet been applied — class imbalance is currently addressed only via `class_weight`-based reweighting in the Random Forest classifier, and extending sample-weighting and/or SMOTE to the remaining classifiers is identified as future work. Mutual-Information-based feature ranking and dimensionality reduction to a reduced feature subset for deep-learning models is likewise deferred until the deep-sequence tier (Tier 2, Section VI-A) is implemented; the classical ensemble tier currently trains on a fixed, manually engineered 46-feature subset of the CICFlowMeter schema.

Processed data is partitioned using stratified sampling to preserve class distributions across training and test subsets.

IX. Experimental Results and Performance Analysis

Note on scope: This section reports measured results only for the components that have been implemented and evaluated to date (the classical detection ensemble). Performance figures for components that remain future work (deep-sequence detection, DQN response, BERT-based threat intelligence, GAN/EWC self-evolution) are reported separately in Table V as literature-derived projections, consistent with their status in Table II, and are not claimed as achieved results.

A. Measured Detection Performance

Table IV reports the measured performance of the classifiers currently trained and integrated into the AI engine, evaluated on a held-out stratified split of the four integrated CICIDS2017 capture files. The Random Forest classifier is trained on real CICIDS2017 traffic; the XGBoost and secondary Isolation Forest variants reported here are trained on the platform's synthetic data generator pending real-data training (see the caveat below the table). False-positive rate, AUC-ROC, and end-to-end response latency have not yet been instrumented and are reported as future measurements rather than estimated, consistent with the honest-reporting position of this paper.

TABLE V
Projected Performance of Components Not Yet Implemented (Literature-Derived, Not Measured)

Component (Future Work)	Projected Result	Source
1D-CNN deep-sequence tier	$\approx 97.8\%$ accuracy	[12]
Full 5-tier ensemble with meta-learner	$\approx 98.7\%$ accuracy	Sec. IX projection
DQN Response Engine	Not applicable	[19] (reward convergence only)

TABLE IV
Measured Detection Performance (CICIDS2017, Real Data unless noted)

Model	Training data	Accuracy	F1 (weighted)
Random Forest	Real CICIDS2017	98.43%	0.9862
Random Forest	Synthetic	93.20%	0.9313
XGBoost	Synthetic	83.67%	0.8348

Two caveats are reported for transparency. First, the current Random Forest artifact reports a macro F1 of 0.8448 despite its high weighted F1 of 0.9862, indicating substantial class-imbalance effects. Accuracy and weighted F1 should therefore not be used alone to characterise operational alert quality. Second, because only six of the eleven target classes currently have real CICIDS2017 training examples (Section VII-A), the reported Random Forest figures should not be interpreted as full eleven-class operational performance. Per-class precision, recall, and false-positive measurements for the complete eleven-class problem remain required. XGBoost has not yet been trained on the real CICIDS2017 split, so its synthetic-data result in Table IV is not directly comparable to the Random Forest real-data result.

B. Projected Performance of Components Under Development

Table V restates, for components that remain future work, the same literature-derived projections as in earlier design documentation. These figures are *not* results from the deployed SentinelAI system; they represent expected performance upon full implementation of the deep-sequence tier, DQN response engine, and BERT-based threat intelligence engine, derived from the cited component-level benchmarks in Table I.

C. Adaptive Honeypot: Implemented but Not Yet Formally Evaluated

The Q-learning-based honeypot profile recommendation mechanism described in Section VI-B and Algorithm 2 is

implemented and running against live honeypot sessions, with a persistent Q-table that is inspectable for audit purposes. The current implementation generates and records alternative honeypot profile recommendations based on observed attacker interaction and fingerprinting behaviour; it does not automatically modify the active Cowrie configuration at runtime.

A formal comparison of attacker engagement duration against a static-honeypot control—the experiment required to substantiate the $\geq 40\%$ engagement-improvement objective in Section IV-B and the projections previously drawn from [5], [7]—has not yet been run. Designing and executing this controlled comparison is identified as a priority experiment in future work (Section XII), since it would provide direct experimental evidence for the effectiveness of the adaptive honeypot recommendation mechanism.

X. Implementation Roadmap

The SentinelAI implementation is structured across nine sequential phases. Status is reported as of this revision.

Phase 1 — Frontend Prototype (Weeks 1–3): Development of the user interface using React.js, Vite, Tailwind CSS, and Recharts for live data visualisation. *Status: Complete.* The dashboard now renders live data streamed from the backend via WebSockets rather than static mock data.

Phase 2 — Backend API Development (Weeks 4–6): Implementation of RESTful services using Node.js and Express.js for telemetry ingestion, threat querying, user management, and automated response coordination. *Status: Complete.* JWT-based authentication with three-tier role-based access control (administrator, analyst, viewer), request rate limiting, and centralised error handling are implemented and enforced across all API routes. A machine-readable OpenAPI/Swagger specification has not yet been generated and is carried forward as a documentation task.

Phase 3 — Database Design and Integration (Weeks 7–8): Design and deployment of MongoDB collections for network telemetry, threat logs, honeypot session data, user accounts, and alerts. *Status: Complete.* Compound indexes matching the platform's real query patterns (e.g., severity/status, source IP/timestamp) are in place; time-series-optimised and geospatial indexing are not yet implemented and remain future refinements as data volume grows.

Phase 4 — Threat Detection Engine (Weeks 9–13): Training and validation of Random Forest, XGBoost, 1D-CNN, Isolation Forest, and Autoencoder models on the designated benchmark datasets. *Status: Partially complete.* Random Forest, XGBoost, Isolation Forest, and an SGD online-learning interface are supplied; the live inference path loads the Random Forest CICIDS v2 artifact together with Isolation Forest. The deployed Random Forest artifact reports 98.43% accuracy and 0.9862 weighted F1 on its evaluation data (Table IV). The 1D-CNN and Autoencoder components, and the learned meta-learner stacking layer, remain to be implemented. Target of validation F1 >0.97 has been met for the implemented Random Forest classifier on its currently covered classes.

Phase 5 — Adaptive Honeypot Intelligence (Weeks 14–17): Implementation of the honeypot orchestration layer and an RL agent for dynamic reconfiguration and deception optimisation. *Status: Complete (Q-learning recommendation variant).* A Cowrie-based SSH honeypot is integrated with a tabular Q-learning agent that generates OS/service profile recommendations based on attacker engagement signals. The active Cowrie configuration is not automatically changed by the current agent, as described in Section VI-B. Migration to a PPO-based policy for larger action spaces remains future work.

Phase 6 — RL Response Engine (Weeks 18–21): Development of the DQN-based autonomous response system for firewall rule deployment, network isolation, and traffic redirection. *Status: Not started.* The current system produces and logs response recommendations and supports operator-triggered logical blocking, but does not yet enforce actions at the network layer. This is the highest-priority remaining phase, as it is the component that would make the platform's automated response genuinely autonomous rather than decision-support.

Phase 7 — Threat Intelligence Integration (Weeks 22–24): Fine-tuning of the BERT-based extraction model, construction of the cybersecurity knowledge graph, and MITRE ATT&CK mapping integration [23]. *Status: Partially complete.* Automated MITRE ATT&CK technique tagging is implemented and applied to every detection; IP-reputation enrichment via AbuseIPDB was added as an additional signal. BERT-based extraction from unstructured CTI sources and the knowledge-graph store remain future work.

Phase 8 — System Testing and Evaluation (Weeks 25–28): End-to-end benchmark testing, ablation studies, penetration testing, scalability analysis, and false-positive rate assessment. *Status: Not started.* No automated test suite or CI/CD pipeline currently exists for any of the four services; this is identified as an immediate priority alongside Phase 6, since it is a precondition for trustworthy ablation and false-positive measurements.

Phase 9 — Production Deployment (Weeks 29–32): Docker containerisation, Kubernetes orchestration, production readiness assessment, and security validation. *Status: Partially complete.* All four services (frontend, backend, AI engine, honeypot) are containerised and orchestrated via Docker Compose, with optional service profiles for the AI engine and honeypot. Kubernetes orchestration and a formal production security review (see Section XII) remain future work.

XI. Prototype Implementation

The current implementation of SentinelAI consists of a functional, integrated four-service platform: a React/Vite frontend, a Node.js/Express backend with MongoDB, a Python/FastAPI AI inference engine, and a Cowrie-based adaptive honeypot, deployed together via Docker Compose. This goes beyond the frontend-only proof-of-concept reported in earlier work: the interfaces described below are now backed by live data and a trained detection pipeline rather than static

mock content, and constitute the corresponding evidence for the roadmap phases marked complete or partially complete in Section X.

A. Operator Interface

The React/Vite frontend provides a SOC-style interface with dashboard, threat-log, AI-analytics, honeypot, campaign, alert, and user-management views. The frontend consumes backend REST APIs and Socket.IO events. The project also contains a mock-data module used by portions of the UI; therefore individual visual elements should only be treated as live when their page actually fetches backend data.

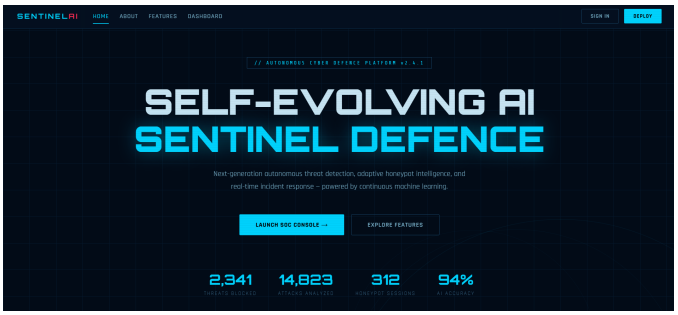


Fig. 2. Homepage of the SentinelAI Platform

B. Authentication and Access Control

The backend implements JWT-based access tokens, bcrypt password hashing, role-based restrictions, and rate-limited authentication endpoints. Operator routes are protected through middleware. However, refresh tokens are currently stateless and token revocation is not implemented. The Socket.IO server accepts a token from the client but does not currently validate it before trusting the client-supplied room role. These are security hardening gaps and should not be described as a complete Zero Trust implementation.

C. SOC Command Centre Dashboard

The Security Operations Center (SOC) dashboard aggregates critical security metrics into a unified operational view, displaying threat statistics, attack trends, security alerts, and system status indicators. Unlike the earlier prototype, this dashboard is now populated with live data pushed from the backend over WebSockets as detections and honeypot events occur, rather than static or simulated content.

D. Honeypot Telemetry

Cowrie provides the SSH interaction environment. The parser captures session creation, failed and successful logins, commands, file downloads, and HASSH fingerprints. Command intents are mapped to ATT&CK techniques, and the backend stores session, IOC, engagement, and malware-download information.

The Cowrie configuration supplied with the project is a fixed Ubuntu-oriented profile. The Q-learning module returns alternative profile recommendations, but no runtime mechanism changes the active Cowrie configuration. The distinction between recommendation and enforcement is central to the current implementation status.

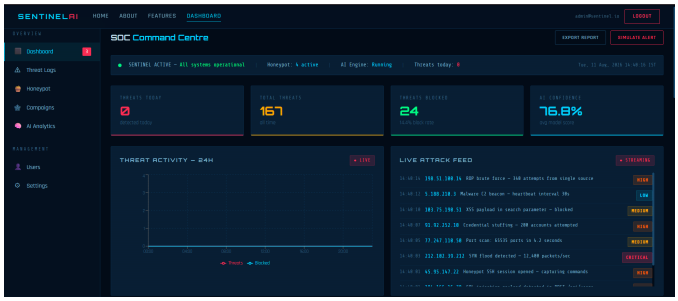


Fig. 3. SOC Command Centre Dashboard

E. Campaign Analysis

The campaign page consumes a backend clustering service that groups sessions using exact HASSH equality or command-set similarity. Drill-down views expose campaign members, source IPs, HASSH values, threat scores, and deception scores. This is an implemented analytical capability and should be included among the paper's contributions.

F. AI Analytics

The AI analytics service exposes model statistics, predictions, anomaly rates, inference-time measurements collected by the application, and class-level prediction counts. SHAP explanations are available for supported tree-based models. The UI also contains static/mock log text and visual content, so these presentation elements should not be treated as independent experimental evidence.

G. Testing and Reproducibility

The backend contains four Jest test files covering authentication, honeypot-session processing, campaign clustering, and MITRE mapping. The supplied environment could not execute the test command because the packaged Jest executable returned a permission error; therefore test-pass status was not independently established in this review. No equivalent automated test suite is supplied for the frontend, AI engine, or honeypot containers.

H. Discussion

The prototype demonstrates that the four SentinelAI services can be integrated end-to-end: live network and honeypot events flow from ingestion through ML inference to an operator-facing dashboard using WebSockets. The honeypot intelligence component also processes real interaction data to generate adaptive profile recommendations rather than relying solely on a fixed policy. However, the active Cowrie configuration remains unchanged automatically in the current implementation; runtime application of the recommended deception profile is reserved for future work.

These results validate the platform's core data-flow design (Section V) in practice, not only on paper. The principal remaining gaps are the ones enumerated in Table II: closing the

autonomous-response loop (Phase 6), broadening real training data to the full eleven-class taxonomy, formally evaluating the adaptive honeypot against a static baseline, and building the automated test suite needed to make the remaining performance claims measurable rather than projected.

XII. Future Scope

Although the current SentinelAI prototype provides an operational four-service architecture and implements substantial portions of the proposed detection, deception, threat-intelligence, and continual-learning pipeline, several components remain to be completed and experimentally validated. The main future directions are as follows.

- **Autonomous Response and Deep Learning:** The detection and recommendation pipeline will be extended to controlled network-layer enforcement through scoped firewall or cloud security-group rules. The proposed DQN-based Response Engine and the 1D-CNN/LSTM deep-sequence detection tier will then be implemented to move SentinelAI toward fully autonomous threat detection and response.
- **Complete Real-Data Evaluation:** The remaining official CICIDS2017 capture files will be integrated to provide sufficient real-data samples for the currently data-sparse attack classes. Random Forest, XGBoost, and the complete ensemble will then be re-evaluated using the resulting multi-class dataset.
- **Adaptive Honeypot Validation:** A controlled experiment will compare the Q-learning-based honeypot profile recommendation mechanism against a static-honeypot baseline using attacker engagement duration, interaction depth, and behavioural observations. This will provide direct experimental evidence for the effectiveness of adaptive deception.
- **Cross-Engine Integration and Explainability:** Future experiments will investigate whether behavioural information obtained from the honeypot improves threat-detection performance through controlled ablation studies. SHAP-based explanations will also be extended to honeypot profile recommendations and autonomous response decisions to improve interpretability and auditability.
- **Threat Intelligence and Self-Evolution:** The current MITRE ATT&CK and IP-reputation components will be extended with BERT-based extraction and knowledge-graph correlation for unstructured threat intelligence. The SGD-based continual-learning mechanism will subsequently be extended toward the proposed meta-learning framework with GAN-based augmentation and Elastic Weight Consolidation.
- **Robustness and Generalisation:** Future evaluation will include adversarial-input validation, label-poisoning resistance, and cross-dataset testing using UNSW-NB15 and CIC-DDoS2019. These experiments will assess the robustness and generalisation of SentinelAI under distribution shifts and adaptive attack conditions.

XIII. Conclusion

This paper presented SentinelAI, an adaptive cyber-defence architecture designed to integrate adaptive honeypot intelligence, multi-model threat detection, DRL-based response orchestration, NLP-driven threat intelligence, and meta-learning self-evolution into a unified five-engine platform. Beyond the architectural design, formal problem formulation, and threat model contributed in earlier work, this revision reports a functional, integrated four-service implementation: a live full-stack platform with JWT/RBAC-secured APIs, a Random Forest detection model validated on real CICIDS2017 traffic at 98.43% accuracy and 0.9862 weighted F1, a Q-learning-based honeypot profile recommendation mechanism with a persistent, auditable policy, automated MITRE ATT&CK tagging, and SHAP-based explainability for detection outputs. These results replace several of the literature-derived projections in earlier work with measured outcomes, while other components of the original five-engine design — the DQN response engine, the BERT-based threat intelligence pipeline, the deep-sequence detection tier, and GAN/EWC-based meta-learning — remain honestly reported as future work rather than claimed as implemented. Closing the autonomous-response loop, completing real-data coverage of the attack taxonomy, and building an automated evaluation suite are identified as the highest-priority next steps toward substantiating the platform's full integration-novelty claim.

References

[1] M. Garg, P. R. Kaza, A. Kilaru, S. S. Basha, C. Gunupudi, and K. K. Kondapalli, "Adaptive AI-Driven Threat Intelligence Framework for Proactive Cyber Defense in Dynamic Network Environments," in *Proc. Int. Conf. AI-Driven Smart Systems and Ubiquitous Computing (ICAUC)*, 2026, pp. 1995–2000, doi: 10.1109/ICAUC68182.2026.11441080.

[2] K. M. Rao and B. Kumar, "An Adaptive Cyber Threat Intelligence System for Monitoring and Countering Underground Economy Operations," in *Proc. Int. Conf. AI-Driven Smart Systems and Ubiquitous Computing (ICAUC)*, 2026, pp. 1552–1557, doi: 10.1109/ICAUC68182.2026.11441318.

[3] M. Ponaganti, O. Khresat, T. Vijetha, S. Sathiya Priya, R. Y. P., and G. Hemalatha, "Adaptive AI Algorithms for Cyber Defense: Predictive Behavioral Analysis to Mitigate Zero-Day Attacks," in *Proc. 5th Int. Conf. Emerging Research in Electronics, Computer Science and Technology (ICERECT)*, 2025, pp. 1–6, doi: 10.1109/ICERECT65215.2025.11377505.

[4] M. N. Abbas and M. N. Asghar, "Adaptive Cyber-defence: A Reinforcement Learning-based Response to the Evolving Cyber-attacks in Network Security," in *Proc. Cyber Research Conference – Ireland (Cyber-RCI)*, 2025, pp. 1–8, doi: 10.1109/Cyber-RCI68134.2025.11385085.

[5] A. Tiwari and D. Kumar, "Adaptive Fuzzy Logic Framework for Threat Detection in Distributed Honeypot Systems Leveraging Blockchain Technology," in *Proc. 2nd Int. Conf. Computational Intelligence, Communication Technology and Networking (CICTN)*, 2025, pp. 45–49, doi: 10.1109/CICTN64563.2025.10932344.

[6] K. Malarvizhi, M. Ravivarman, R. Chandru, S. Dharshan, M. Ranjith, and S. Nagaraj, "AI Powered Honeypots for Advanced Threat Detection and Analysis," in *Proc. 5th Int. Conf. Evolutionary Computing and Mobile Sustainable Networks (ICECMSN)*, 2025, pp. 1985–1994, doi: 10.1109/ICECMSN68058.2025.11382640.

[7] R. Alosaimi and A. Alatawi, "Leveraging Artificial Intelligence for Enhancing Real-Time Honeypot Security," in *Proc. 4th Int. Conf. Computing and Information Technology (ICCIT)*, 2025, pp. 717–722, doi: 10.1109/ICCIT63348.2025.10989357.

[8] A. K. Polinati, "Generative Adversarial Deception Networks (GADN): Self-Evolving AI for Adaptive Cyber Defense in Cloud-Native Security Architectures," in *Proc. 10th Int. Conf. Science Technology Engineering and Mathematics (ICONSTEM)*, 2025, pp. 1–8, doi: 10.1109/ICONSTEM65670.2025.11374768.

[9] M. I. M. Tanvir, N. Y. Nadia, H. R. Rabby, M. H. Arif, S. R. Sultana, and K. Nur, "Self-Supervised and Domain-Adaptive Deep Learning for Early Detection of Cyber-Attacks in Healthcare IoT Systems," in *Proc. Int. Conf. Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT)*, 2025, pp. 1–8, doi: 10.1109/3ICT68299.2025.11442237.

[10] M. Faísco, I. Medeiros, and H. P. Reiser, "Towards Real-Time Malware Classification Through Honeypot Analysis," in *Proc. 20th European Dependable Computing Conference Companion Proceedings (EDCC-C)*, 2025, pp. 56–59, doi: 10.1109/EDCC-C66476.2025.00030.

[11] A. Altrad, A. Soy, D. Najmutdinova, M. Mahfuri, Q. R. H. Al-Zoubi, and K. Sanjay, "Random Forest-Based Intrusion Detection for Zero-Day Attack Mitigation in Industrial Control Systems," in *Proc. Int. Conf. Electrical Engineering and Informatics (ICEEI)*, 2025, pp. 1–7, doi: 10.1109/ICEEI68459.2025.11331053.

[12] Y. Wang, Q. Liu, P. Liang, and C. Sui, "Research on Real-time DDoS Attack Detection System Based on One-dimensional Deep Convolutional Neural Network," in *Proc. Int. Conf. Algorithms, Software and Network Security (ASNS)*, 2025, pp. 74–77, doi: 10.1109/ASNS67347.2025.11345720.

[13] M. Bouriche, O. Iraqi, and H. El Bakkali, "Neural Hidden Markov Models for Semi-Supervised Detection of Brute Force Attacks," in *Proc. 5th Int. Conf. Innovative Research in Applied Science, Engineering and Technology (IRASET)*, 2025, pp. 1–5, doi: 10.1109/IRASET64571.2025.11008119.

[14] K. Z. Htet, H. T. Zaw, and A. H. Maw, "Brute Force Detection and Active Response for Secure Web Login Using Wazuh," in *Proc. 6th Int. Conf. Advanced Information Technologies (ICAIT)*, 2025, pp. 1–6, doi: 10.1109/ICAIT68809.2025.11236796.

[15] N. Zahreldine, A. Jaber, A. H. El Fawal, D. M. Bleik, and A. Mansour, "Live Detection and Mitigation for SSH Brute Force Attacks," in *Proc. IEEE Int. Conf. Emerging Trends in Engineering and Computing (ETECOM)*, 2025, pp. 1–5, doi: 10.1109/ETECOM66111.2025.11319006.

[16] Y. Wang, A. Habib, L. R. Kesara, B. Jasti, R. Hu, and T. Singh, "Detecting Black-Box Model Probing Attacks Through Probability Scores," *IEEE Access*, vol. 13, pp. 103060–103076, 2025, doi: 10.1109/ACCESS.2025.3578565.

[17] J. F. Loevenich et al., "Towards Robust and Secure Autonomous Cyber Defense Agents in Coalition Networks," in *Proc. MILCOM 2024 – IEEE Military Communications Conference*, 2024, pp. 152–157, doi: 10.1109/MILCOM61039.2024.10773821.

[18] J. H. Al-Yasiri, M. F. Bin Zolkipli, N. F. N. M. Farid, M. Alsamman, and Z. A. Mohammed, "A Threat Intelligence Event Extraction Conceptual Model for Cyber Threat Intelligence Feeds," in *Proc. 7th Int. Conf. Internet Applications, Protocols, and Services (NETAPPS)*, 2024, pp. 1–8, doi: 10.1109/NETAPPS63333.2024.10823639.

[19] A. Babar, L. Li, A. Taylor, and M. Zulkernine, "Towards Autonomous Network Defense: Reinforcement Learning Environment for a Defense Agent," in *Proc. IEEE CCECE*, 2024, pp. 168–169, doi: 10.1109/CCECE59415.2024.10667139.

[20] P. Théron and A. Kott, "When Autonomous Intelligent Goodware Will Fight Autonomous Intelligent Malware: A Possible Future of Cyber Defense," in *Proc. MILCOM*, 2019, pp. 1–7, doi: 10.1109/MILCOM47813.2019.9020847.

[21] IBM Security, "Cost of a Data Breach Report 2023," Tech. Rep., 2023. Available: https://www.ibm.com/reports/data-breach

[22] Mandiant, "M-Trends 2023: Special Report," Tech. Rep., 2023. Available: https://www.mandiant.com/m-trends

[23] MITRE Corporation, "MITRE ATT&CK: Design and Philosophy," Tech. Rep. MTR190811, 2023. Available: https://attack.mitre.org

[24] T. Holz and F. Raynal, "Detecting Honeypots and Other Suspicious Environments," in *Proc. IEEE Workshop on Information Assurance and Security*, 2005, pp. 29–36, doi: 10.1109/IAW.2005.1495930.

[25] G. Apruzzese, M. Colajanni, L. Ferretti, A. Guido, and M. Marchetti, "On the Effectiveness of Machine and Deep Learning for Cyber Security," in *Proc. 10th Int. Conf. Cyber Conflict (CyCon)*, 2018, pp. 371–390, doi: 10.23919/CYCON.2018.8405026.

[26] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A Survey of Network-based Intrusion Detection Data Sets," *Computers & Security*, vol. 86, pp. 147–167, 2019, doi: 10.1016/j.cose.2019.06.005.

[27] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.

[28] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[29] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.